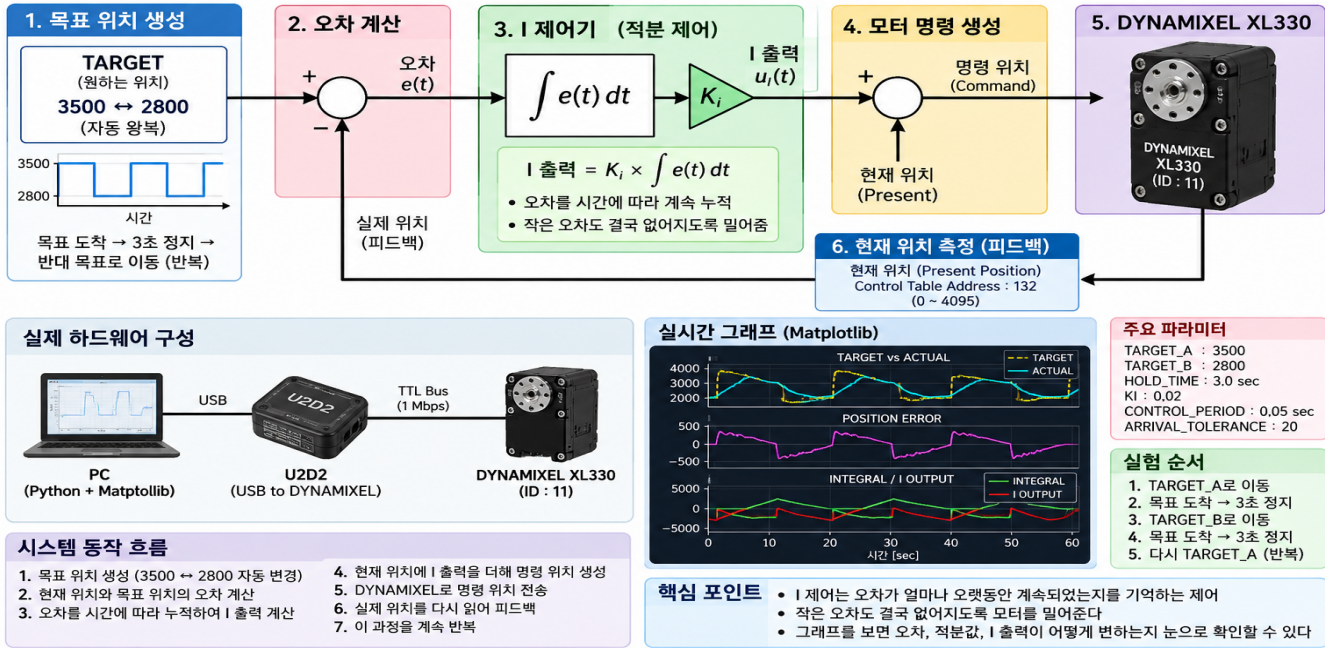


DYNAMIXEL XL330으로 배우는 I 제어

적분 제어의 기초부터 Python 실험, 실시간 그래프, CSV 기록까지

DYNAMIXEL XL330 I 제어 전체 시스템 설계도

목표 위치와 실제 위치의 오차를 시간에 따라 누적하여 모터를 목표 위치로 이동시키는 시스템



실험 환경: Ubuntu / Python / U2D2 / DYNAMIXEL XL330 / Matplotlib / CSV

1. 오늘의 학습 목표

이번 실험의 목적은 DYNAMIXEL XL330 한 개를 실제로 움직이면서 I 제어(Integral Control)의 원리를 눈으로 확인하는 것입니다.

전체 흐름은 PC → U2D2 → DYNAMIXEL XL330으로 구성하고, Python과 DYNAMIXEL SDK로 모터 위치를 읽고 명령합니다. Matplotlib으로 Target, Actual, Error, Integral, I Output을 실시간 표시하고 CSV로 데이터를 저장합니다.

2. I 제어란 무엇인가

P 제어가 '현재 오차'에 반응한다면 I 제어는 '오차가 얼마나 오랫동안 남아 있었는가'를 기억합니다.

핵심 식은 $Error = Target - Actual$, $Integral = Integral + Error \times dt$, $I\ Output = K_i \times Integral$ 입니다.

작은 오차라도 오래 지속되면 적분값이 커지고 제어 출력이 증가합니다. 따라서 정상상태 오차를 줄이는 데 유용하지만, 너무 큰 K_i 나 누적 오차는 오버슈트와 진동을 만들 수 있습니다.

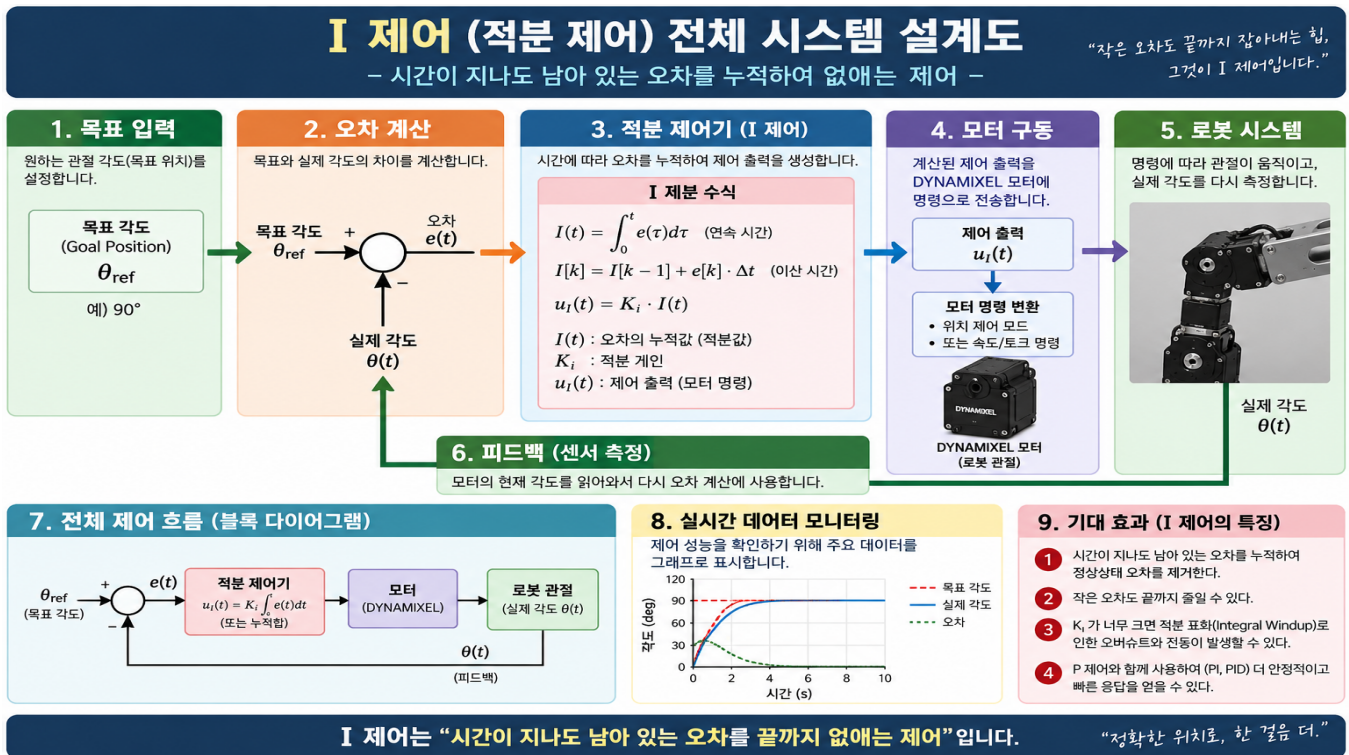


그림 1. I 제어 개념 및 실험 흐름

3. 실험 시스템 구성

하드웨어: Ubuntu PC, U2D2, DYNAMIXEL XL330 - M288 - T, 전원 및 TTL 케이블.

소프트웨어: Python 3, dynamixel-sdk, matplotlib, csv.

실험 기준 설정: /dev/ttyUSB0, 1,000,000 bps, Protocol 2.0, DXL ID 11, 목표 위치 3500 ↔ 2800, 도착 후 3초 정지.

DYNAMIXEL XL330 I 제어 전체 시스템 설계도

목표 위치와 실제 위치의 오차를 시간에 따라 누적하여 모터를 목표 위치로 이동시키는 시스템

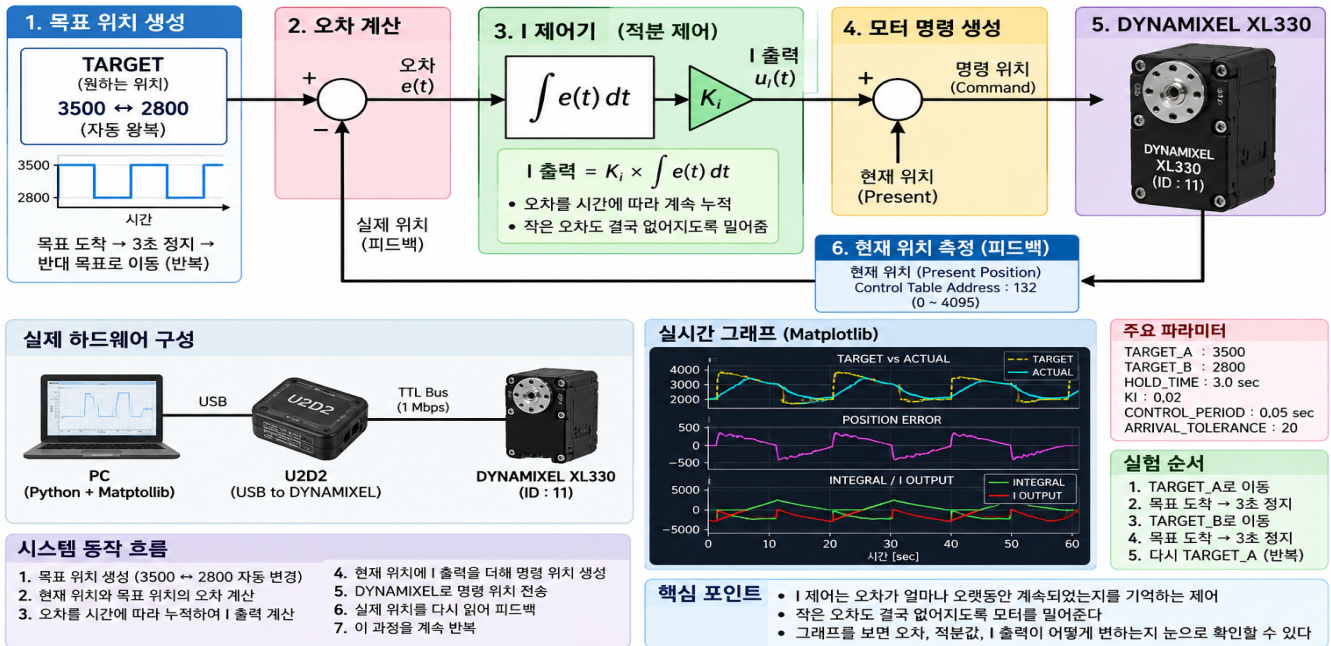


그림 2. DYNAMIXEL XL330 I 제어 전체 시스템 설계도

4. 터미널에서 처음부터 시작하기

- 장치 확인: `ls /dev/ttyUSB*`
- 라이브러리 설치: `python3 -m pip install dynamixel-sdk matplotlib`
- 홈 폴더 이동: `cd ~`
- 파일 생성: `nano i_control_graph_test.py`
- 코드 저장: Ctrl+O → Enter, 편집기 종료: Ctrl+X
- 실행: `python3 ~/i_control_graph_test.py`
- 종료: Ctrl+C
- CSV 확인: `ls -lh ~/i_control_*.csv`

5. Python 코드의 구조

코드는 라이브러리 불러오기 → 통신 설정 → Control Table 주소 설정 → 목표/제어 파라미터 설정 → CSV 준비 → U2D2 포트 열기 → Torque ON → 그래프 준비 → while 제어 루프 → 안전 종료 순으로 구성됩니다.

제어 루프에서는 Present Position을 읽고 Error를 계산한 뒤 Integral을 갱신하고 I Output을 계산합니다. 계산한 출력으로 새로운 Goal Position을 만들어 XL330으로 전송하고, 다음 루프에서 다시 실제 위치를 읽습니다.

6. 핵심 제어 코드

```
position_error = TARGET - position
integral += position_error * dt
i_output = KI * integral
command = int(position + i_output)
```

이 네 줄이 실험의 핵심입니다. 특히 $\text{integral} += \text{error} \times \text{dt}$ 와 $i_{\text{output}} = K_i \times \text{integral}$ 이 I 제어를 직접 구현한 부분입니다.

7. Anti - Windup

모터가 움직이지 못하는 동안 오차가 계속 누적되면 Integral이 지나치게 커질 수 있습니다. 이를 Integral Windup이라고 합니다.

실험 코드에서는 INTEGRAL_LIMIT와 I_OUTPUT_LIMIT를 두어 누적값과 출력값을 제한합니다. 이것은 교육용으로 이해하기 쉬운 기본 Anti - Windup 방법입니다.

8. 자동 왕복 운동

TARGET_A=3500, TARGET_B=2800을 사용합니다. 목표 ± 20 이내에 들어오면 도착으로 판단하고 HOLD 상태에서 3초 기다립니다. 이후 목표를 반대로 바꾸고 다시 MOVE 상태로 들어갑니다.

따라서 3500 → 도착 → 3초 정지 → 2800 → 도착 → 3초 정지 → 3500의 과정이 자동 반복됩니다.

9. 실시간 그래프 읽는 법

TARGET vs ACTUAL: 목표 위치와 실제 모터 위치가 얼마나 잘 일치하는지 봅니다.

POSITION ERROR: 목표와 실제 위치의 차이가 시간이 지나며 0에 가까워지는지 봅니다.

INTEGRAL / I OUTPUT: 오차가 누적되는 과정과 그 누적값이 실제 제어 출력으로 변환되는 과정을 봅니다.

10. CSV 기록의 의미

시간, 목표 위치, 실제 위치, 오차, 적분값, I 출력, 명령 위치, 상태를 CSV로 기록합니다.

이 데이터를 이용하면 K_i 를 바꿨을 때 응답 속도, 오버슈트, 오차 감소 특성이 어떻게 변하는지 비교할 수 있습니다.

11. 중요한 기술적 주의점

이번 실험은 XL330의 내부 Position Mode 위에 Python의 교육용 외부 I 루프를 한 겹 더 얹은 구조입니다. 따라서 모터 내부의 저수준 제어를 완전히 I 제어만으로 교체한 실험은 아닙니다.

처음에는 부하를 제거한 상태에서 낮은 K_i 로 시험하고, 위치 범위 0~4095를 넘는 명령을 보내지 않도록 제한해야 합니다.

12. 오늘의 결론과 다음 단계

로봇 제어의 기본은 목표값을 정하고 실제 상태를 측정하고 오차를 계산한 뒤 새로운 명령을 보내는 피드백 반복입니다.

P는 현재 오차, I는 누적된 과거 오차, D는 오차의 변화 속도를 봅니다. 다음 단계에서는 P와 I를 결합한 PI 제어를 같은 XL330 실험에 적용해 응답 차이를 비교할 수 있습니다.

부록 A. 전체 Python 실험 코드

```
#!/usr/bin/env python3
import time, csv
from datetime import datetime
import matplotlib.pyplot as plt
from dynamixel_sdk import PortHandler, PacketHandler, COMM_SUCCESS

DEVICENAME = "/dev/ttyUSB0"
BAUDRATE = 1000000
PROTOCOL_VERSION = 2.0
DXL_ID = 11

ADDR_TORQUE_ENABLE = 64
ADDR_GOAL_POSITION = 116
ADDR_PRESENT_POSITION = 132
TORQUE_ENABLE = 1
TORQUE_DISABLE = 0

TARGET_A = 3500
TARGET_B = 2800
TARGET = TARGET_A

KI = 0.02
CONTROL_PERIOD = 0.05
INTEGRAL_LIMIT = 5000.0
I_OUTPUT_LIMIT = 150.0
ARRIVAL_TOLERANCE = 20
HOLD_TIME = 3.0
POSITION_MIN = 0
POSITION_MAX = 4095

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
CSV_FILE = f"i_control_{timestamp}.csv"
csv_file = open(CSV_FILE, "w", newline="")
csv_writer = csv.writer(csv_file)
csv_writer.writerow(["time", "target", "actual", "error", "integral", "i_output", "command", "state"])

portHandler = PortHandler(DEVICENAME)
packetHandler = PacketHandler(PROTOCOL_VERSION)

if not portHandler.openPort():
    raise RuntimeError(f"PORT OPEN FAILED: {DEVICENAME}")
if not portHandler.setBaudRate(BAUDRATE):
    raise RuntimeError("BAUDRATE SET FAILED")

packetHandler.write1ByteTxRx(portHandler, DXL_ID, ADDR_TORQUE_ENABLE, TORQUE_ENABLE)

plt.ion()
fig, axes = plt.subplots(3, 1, figsize=(12, 9))
line_target, = axes[0].plot([], [], "--", label="TARGET")
line_actual, = axes[0].plot([], [], label="ACTUAL")
axes[0].set_title("TARGET vs ACTUAL"); axes[0].grid(True); axes[0].legend()
line_error, = axes[1].plot([], [], label="ERROR")
axes[1].axhline(0, linestyle="--"); axes[1].set_title("POSITION ERROR"); axes[1].grid(True); axes[1].legend()
line_integral, = axes[2].plot([], [], label="INTEGRAL")
line_output, = axes[2].plot([], [], label="I OUTPUT")
```

```

axes[2].axhline(0, linestyle="--"); axes[2].set_title("INTEGRAL / I OUTPUT"); axes[2].grid(True); axes[2].legend()
plt.tight_layout()

time_data=[]; target_data=[]; actual_data=[]; error_data=[]; integral_data=[]; output_data=[]
integral = 0.0
start_time = time.monotonic()
previous_time = start_time
state = "MOVE"
hold_start_time = None

try:
    while True:
        loop_start = time.monotonic()
        elapsed_time = loop_start - start_time
        dt = loop_start - previous_time
        previous_time = loop_start
        if dt <= 0: dt = CONTROL_PERIOD

        position, result, dxl_error = packetHandler.read4ByteTxRx(
            portHandler, DXL_ID, ADDR_PRESENT_POSITION
        )
        if result != COMM_SUCCESS or dxl_error != 0:
            time.sleep(CONTROL_PERIOD)
            continue

        position_error = TARGET - position

        if state == "MOVE":
            integral += position_error * dt
            integral = max(-INTEGRAL_LIMIT, min(INTEGRAL_LIMIT, integral))
            i_output = KI * integral
            i_output = max(-I_OUTPUT_LIMIT, min(I_OUTPUT_LIMIT, i_output))
            command = int(position + i_output)
            command = max(POSITION_MIN, min(POSITION_MAX, command))
            packetHandler.write4ByteTxRx(portHandler, DXL_ID, ADDR_GOAL_POSITION, command)

            if abs(position_error) <= ARRIVAL_TOLERANCE:
                state = "HOLD"
                hold_start_time = loop_start
                integral = 0.0
        else:
            i_output = 0.0
            command = TARGET
            packetHandler.write4ByteTxRx(portHandler, DXL_ID, ADDR_GOAL_POSITION, command)
            if loop_start - hold_start_time >= HOLD_TIME:
                TARGET = TARGET_B if TARGET == TARGET_A else TARGET_A
                integral = 0.0
                state = "MOVE"

        print(f"T={elapsed_time:7.2f} | {state:4s} | TARGET={TARGET:4d} | "
              f"ACTUAL={position:4d} | ERROR={position_error:5d} | "
              f"INT={integral:9.2f} | I_OUT={i_output:7.2f} | CMD={command:4d}")

        csv_writer.writerow([elapsed_time, TARGET, position, position_error, integral, i_output, command, state])
        csv_file.flush()

```

```

time_data.append(elapsed_time); target_data.append(TARGET); actual_data.append(position)
error_data.append(position_error); integral_data.append(integral); output_data.append(i_output)
line_target.set_data(time_data, target_data); line_actual.set_data(time_data, actual_data)
line_error.set_data(time_data, error_data); line_integral.set_data(time_data, integral_data)
line_output.set_data(time_data, output_data)
xmin=max(0,elapsed_time-60); xmax=max(10,elapsed_time)
for ax in axes:
    ax.set_xlim(xmin,xmax); ax.relim(); ax.autoscale_view(scalex=False,scaley=True)
fig.canvas.draw_idle(); fig.canvas.flush_events(); plt.pause(0.001)

sleep_time = CONTROL_PERIOD - (time.monotonic() - loop_start)
if sleep_time > 0: time.sleep(sleep_time)

except KeyboardInterrupt:
    print("I CONTROL TEST STOP")
finally:
    packetHandler.write1ByteTxRx(portHandler, DXL_ID, ADDR_TORQUE_ENABLE, TORQUE_DISABLE)
    portHandler.closePort()
    csv_file.close()
    print(f"CSV SAVED : {CSV_FILE}")
    plt.ioff()
    plt.show()

```